

EIN092B: Laboratorio de Técnicas de Visualización y Agrupación de Datos

Guía de Actividades - Sesión Evaluada

12 de septiembre de 2025

1. Objetivos

- Explorar conjuntos de datos usando técnicas de visualización, agrupación de datos y reducción de la dimensionalidad.
- Entender los conceptos asociados a las técnicas PCA, K-Means, y t-SNE, resaltando las ventajas y desventajas de cada una.

2. Indicaciones generales.

Las actividades planteadas en esta guía deben ser entregadas durante la sesión de laboratorio.

2.1. Sobre el proceso de revisión de actividades.

Para el proceso de revisión consideren lo siguiente:

- La revisión y evaluación de cada una de las actividades se realizará en dos etapas: (i) una validación básica de requerimientos funcionales mínimos durante la sesión, y (ii) una verificación de requerimientos funcionales y no funcionales posterior a la sesión.
- Durante la sesión, avise al staff (profesor o ayudante) cuando complete una o varias actividades para que muestre y explique sus resultados, y responda las preguntas que se le realizarán sobre los conceptos asociados.
- Cualquier integrante del grupo debe ser capaz de responder las preguntas. Si las respuestas no son satisfactorias, o algún integrante no es capaz de responder, no se revisará la actividad hasta que demuestren que TODOS los integrantes del grupo demuestren un entendimiento mínimo sobre lo que están entregando.
- Durante la sesión, el staff marcará en una planilla cuando Ud. considere una actividad terminada y haya pasado la interrogación. El tener la actividad marcada como revisada en la sesión no entrega puntaje, y solo asegura que cumple con requerimientos para que entregue sus archivos y pase a la siguiente etapa de revisión. El formato de entrega de sus archivos se indica en la Sección 2.2.

- En el caso de las actividades no entregadas en la sección correspondiente, contará como no entregada a tiempo y **deberá entregar la actividad corregida en la siguiente sesión, aplicando descuentos por atraso (penalización correspondiente al 35 % del puntaje inicial si se entrega en la siguiente sesión. Si se entrega en dos sesiones posteriores, 50 %).**
- En el caso de que algún integrante del grupo no este presente cuando su grupo correspondiente presente la evaluación y su correspondiente interrogación, deberá presentarlo por su propia cuenta en las siguientes sesiones y sufriendo la penalización por atraso.

2.2. Indicaciones de formato para entrega.

La evaluación constará de dos partes. La primera parte consiste en una validación básica y preguntas sobre los códigos propuestos por el grupo. La segunda parte constará de una verificación de sus códigos y respuestas, la cual se realizará sobre los archivos que deberá entregar vía Aula USM, los cuales deben seguir estrictamente las especificaciones que se le indican a continuación:

1. Debe crear una carpeta con el nombre *GXX_Sesion4*, donde *XX* el número de grupo. Por ejemplo, la carpeta para el grupo 5 se llamaría *G5_Sesion4*. El número de su grupo fue definido en la sesión anterior.
2. Dentro de esta carpeta, debe incluirse un archivo de Jupyter Notebook que contenga el desarrollo de las actividades y las respuestas correspondientes.

Recordar que **será responsabilidad del grupo entregar de manera correcta los archivos**. Se permitirá el envío de los archivos hasta las **23:59 del 12 de Septiembre**. Sin embargo, solo se revisarán las actividades marcadas como revisadas durante la sesión. Cualquier actividad adicional no será considerada en la revisión.

3. Introducción al aprendizaje no supervisado

En cursos anteriores, se ha hecho énfasis principalmente en el uso de métodos de aprendizaje supervisado para resolver problemas de regresión y clasificación. Sin embargo, en gran parte de los problemas no se conoce una variable de respuesta o una etiqueta ante un conjunto de características o mediciones. Bajo este escenario, dado un conjunto de características $X_1, X_2, \dots, X_p$ adquiridas o medidas en n observaciones, el objetivo es encontrar tendencias que proporcionen información sobre el conjunto de mediciones. ¿Qué técnicas permiten visualizar los datos resaltando las características o descubriendo subgrupos dentro de las mediciones? El aprendizaje no supervisado abarca un conjunto diverso de técnicas para resolver interrogantes como este, dado que usualmente se emplea para hacer análisis exploratorio de datos.

En la mayor parte de esta guía de laboratorio, nos enfocaremos en dos tipos particulares de aprendizaje no supervisado: (i) *análisis de componentes principales* (PCA), una herramienta utilizada para la visualización de datos o para el preprocesamiento de datos antes de aplicar técnicas de aprendizaje supervisado, y (ii) *agrupamiento o clustering*, empleando K-Means para encontrar subgrupos no conocidos en el conjunto de datos. Por otra parte, exploraremos el uso de t -SNE para visualización de datos [1].

4. Actividades

En las siguientes subsecciones se describen las actividades a realizar. En cada actividad encontrará indicaciones generales sobre el conjunto de datos a utilizar e indicaciones generales. En este laboratorio, se emplearán técnicas de visualización de datos de alta dimensionalidad y herramientas estadísticas para la agrupación de datos.

4.1. PCA (30 Puntos)

PCA es una herramienta fundamental para el análisis de datos en alta dimensión. Se define formalmente como una transformación lineal ortogonal que mapea los datos a un nuevo sistema de coordenadas, de tal manera que la mayor varianza de cualquier proyección en los datos se encuentra en la primera componente, la segunda mayor varianza en la segunda componente, y así sucesivamente. PCA se clasifica como una técnica de aprendizaje no supervisado, y es comúnmente utilizada para la compresión de datos sin pérdidas significativas¹, la eliminación de redundancias en imágenes y la selección de características, lo que resulta útil en distintos campos.

Existen múltiples enfoques para mapear los datos a un nuevo sistema de coordenadas. El enfoque más común consiste en calcular los vectores y valores propios de la matriz de covarianza, la cual se computa a partir de las variables de entrada. Sin embargo, también existen otros métodos, como la *descomposición en valores singulares* (SVD) [2]. La elección del método para realizar el mapeo comúnmente depende de la capacidad de cómputo disponible y el tamaño de las matrices. En la mayoría de los casos, los usuarios se abstraen de los detalles de bajo nivel que esconden las bibliotecas y pueden realizar la transformación sin necesidad de comprender todos los detalles (simplifica el trabajo cuando se conoce el funcionamiento de la técnica).

La primera actividad se enfoca en el uso de PCA, empleando el *USArrests* dataset. Este dataset contiene el número de detenciones por cada 100.000 residentes para cada uno de los estados de Estados Unidos, y se especifica el número de detenciones para los delitos de agresión, asesinato y violación. También se registra la variable `UrbanPop` que corresponde al porcentaje de la población de cada estado que vive en zonas urbanas.

¹Un ejemplo práctico de esto es el método de *Eigenfaces* (ejemplo de clase), que se utiliza en el reconocimiento facial.

Actividades a realizar:

1. Cargue el conjunto de datos en un dataframe. Extraiga la etiqueta de los datos y las imágenes correspondientes.

```
1 import pandas as pd
2
3 data = pd.read_csv(PATH.DATA)
4 y_train = data.iloc[:, 0].values
5 x_train = data.iloc[:, 1:].values
6
7 print("Dimensiones de x_train:", x_train.shape)
8 print("Dimensiones de y_train:", y_train.shape)
```

Seguidamente, estandarizar cada variable para que tenga media cero y desviación estándar uno. Para un conjunto de datos $\mathbf{X}$, se estandarizan los datos de la siguiente forma:

$$\mathbf{X} = \frac{\mathbf{X} - \mu\mathbf{X}}{\sigma\mathbf{X}}, \quad (1)$$

donde $\mu\mathbf{X}$ y $\sigma\mathbf{X}$ corresponden a la media y la desviación estándar de $\mathbf{X}$, respectivamente.

2. Transforme los datos con la técnica PCA (componentes $n = 2$). Reporte la varianza contenida en las dos primeras componentes.
3. Realice un gráfico de dispersión usando las dos primeras componentes. Además, sobre este gráfico añada los vectores propios; para esto se requiere dibujar un vector desde el origen del plano hasta las coordenadas en los vectores propios. Estas coordenadas indican las direcciones de máxima varianza y muestran cómo las variables originales (Murder, Assault, UrbanPop y Rape) contribuyen a cada componente principal.

Las funciones `pca.explained_variance_` y `pca.components_` permiten obtener los valores y vectores propios, respectivamente. A continuación, se proporciona un código de referencia para hacer una gráfica de los vectores propios de la transformación e incluir el nombre de cada variable. De forma similar, para el gráfico de dispersión de los datos transformados, incluya el nombre de cada estado sobre las coordenadas en el plano cartesiano.

```
1 eigenvectors = pca.components_
2 for i in range(4):
3     plt.quiver(0, 0, eigenvectors[0, i], eigenvectors[1, i],
4               angles='xy', scale_units='xy', scale=1, color='r')
5
6 plt.title('PCA: Datos transformados y Eigenvectores')
7
8 for i, word in enumerate(['Murder', 'Assault', 'UrbanPop', 'Rape']):
9     plt.annotate(word, (eigenvectors[0, i], eigenvectors[1, i]))
```

4. Discuta sobre qué variables tienen mayor impacto en las direcciones de mayor varianza en los datos. ¿Por qué crees que Murder, Assault y Rape están ubicadas cerca unas de otras en el espacio de los componentes principales? ¿Qué indica esto sobre la relación entre estas variables? ¿Qué podría significar el hecho de que UrbanPop esté más lejos de las otras tres variables?

4.2. K-Means Clustering (30 puntos)

K-Means es un enfoque simple para particionar un conjunto de datos en K distintos *clusters* que no se solapan. Cada cluster tiene un centroide que corresponde al promedio de puntos en ese grupo. El objetivo de K-Means es encontrar los K *clusters* que agrupan las observaciones similares entre sí, usando como criterio la distancia al centroide más cercano. Luego, actualizar los centroides como el promedio de los puntos asignados a cada cluster. Este proceso se repite hasta que las asignaciones no cambien o el movimiento de los centroides sea mínimo.

Una debilidad de K-Means es la sensibilidad a la inicialización de los centroides, donde una mala inicialización puede conducir a soluciones subóptimas. Por lo tanto, se han propuesto variantes para mitigar la inicialización (por ejemplo, K-Means++). Para mayores detalles del funcionamiento de K-Means, se provee una presentación en Aula USM.

Si bien existen distintas métricas para evaluar el agrupamiento realizado, como las métricas homogeneidad, completitud, inercia o el índice de *Silhouette*. En este laboratorio, evaluaremos la calidad de la agrupación por *clusters* usando inercia.

El algoritmo de K-Means requiere la integración de algunos pasos generales:

1. Seleccionar el número de *clusters* y asignar aleatoriamente K observaciones del conjunto de datos como centroides.
2. Iterar hasta que las asignaciones de *clusters* no cambien.
 - a) Asignar todos los puntos al centroide más cercano. Seguidamente, para cada uno de los K *clusters* calcule el centroide.
 - b) Asignar cada observación al *clusters* con el centroide más cercano (aquí más cercano es definido usando la distancia euclidiana).

En esta sección utilizaremos el dataset *Fashion-MNIST* conformado por 60.000 imágenes de artículos. Cada imagen contenida en el dataset, esta representada con una profundidad de pixel de 28×28 y asociada a una etiqueta de entre 10 clases disponibles.

Actividades a realizar en esta subsección:

- Implemente el algoritmo K-Means usando la librería *scikit-learn*. En este caso, conocemos que el dataset tiene 10 clases, por lo que se espera que el número optimo de *clusters* sea $K = 10$. De todas formas, esto se hizo intencionalmente para guiar el laboratorio, y poder comparar el agrupamiento obtenido con K-Means con la referencia.
- Realizar un benchmarking utilizando K-Means con una inicialización aleatoria y con una inicialización `KMeans++`. Además, use K-Means con una etapa previa de reducción de la dimensionalidad usando PCA.

Debe considerar que K-Means es un método estocástico por lo que se recomienda ejecutar la inicialización más de una vez, en este caso establezca el parámetro `n_init` en 4. Por el contrario, PCA es un método determinista por lo que puede establecer `n_init` en 1.

```
1 from sklearn.cluster import KMeans
2 from sklearn.decomposition import PCA
```

```

3
4 kmeans = KMeans(init="k-means++", n_clusters=n_classes, n_init=4,
    random_state=0)
5 kmeans = KMeans(init="random", n_clusters=n_classes, n_init=4, random_state
    =0)
6 pca = PCA(n_components=n_classes).fit(data)
7 kmeans = KMeans(init=pca.components_, n_clusters=n_classes, n_init=1)

```

Para evaluar el ajuste de los clusters use como referencia la suma de las distancias al cuadrado (*sum of squared errors*, SSE) de cada punto de datos a su centroide más cercano (atributo **kmeans.inertia_**).

- Evalué los cluster obtenidos en PCA + K-Means visualizando los centroides o visualizando los grupos formados en 2D. En este caso, establezca el número de componentes de PCA en 2.
- Finalmente, para K-Means con inicialización aleatoria, pruebe con distintos valores de K y haga un gráfico donde se muestre el SSE vs. el número de clusters. Discuta los resultados.

4.3. Visualización de datos usando t-SNE (40 puntos)

La visualización de datos en alta dimensionalidad es un problema importante en muchos dominios y se ocupa para visualizar conjuntos de datos de dimensionalidad muy variada. En esta sección, nos enfocaremos en una técnica llamada t-SNE, una herramienta que suele emplearse para la visualización de datos de similitud, la cual es capaz de retener la estructura local de los datos mientras revela alguna estructura global de los datos. t-SNE permite obtener una intuición sobre cómo se organizan los datos en dimensiones superiores. Comúnmente, se usa para visualizar conjuntos de datos complejos en dos y tres dimensiones, lo que nos permite comprender sobre los patrones subyacentes y las relaciones existentes en los datos. Por lo tanto, se podría categorizar como una técnica para explorar un determinado conjunto de datos a fin de encontrar relaciones subyacentes, sin requerir el conocimiento previo de la clase o categoría de cada observación.

t-SNE es definida por sus autores como una técnica para crear un mapa que muestre la estructura de los datos a múltiples escalas diferentes. Particularmente, esto es importante para conjuntos de alta dimensión, como imágenes de objetos de múltiples clases vistos desde múltiples puntos de vista. Generalmente, se emplea el dataset MNIST para realizar evaluaciones preliminares; sin embargo, en esa sección emplearemos el dataset *Olivetti faces*.

Como primer paso, cargue el dataset. El conjunto de datos *Olivetti faces* consta de imágenes de 40 individuos con pequeñas variaciones en el punto de vista, grandes variaciones en la expresión y la adición ocasional de gafas. Este conjunto de datos contiene un conjunto de imágenes de caras tomadas entre abril de 1992 y abril de 1994 en *AT&T Laboratories Cambridge*. El conjunto de datos consta de 400 imágenes (10 por individuo) de tamaño $92 \times 112 = 10.304$ píxeles, y está etiquetado según la identidad. En la Figura 1 se muestran algunos ejemplos de las imágenes que conforman el conjunto de datos.

A continuación, se proporciona un fragmento de código para cargar el dataset ²:

```
1 import logging
```

²Extraído del enlace: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetcholivetti_faces.html



(a) Imágenes extraídas arbitrariamente del conjunto de datos.



(b) Imágenes centradas en la media.

Figura 1: Ejemplos del conjunto de datos de *Olivetti faces*.

```

2
3 import matplotlib.pyplot as plt
4 from numpy.random import RandomState
5
6 from sklearn import cluster, decomposition
7 from sklearn.datasets import fetch_olivetti_faces
8
9 rng = RandomState(0)
10
11 # Display progress logs on stdout
12 logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(
13     message)s")
14
15 faces, _ = fetch_olivetti_faces(return_X_y=True, shuffle=True, random_state=rng)
16 n_samples, n_features = faces.shape
17
18 # Global centering (focus on one feature, centering all samples)
19 faces_centered = faces - faces.mean(axis=0)
20
21 # Local centering (focus on one sample, centering all features)
22 faces_centered -= faces_centered.mean(axis=1).reshape(n_samples, -1)
23
24 print("Dataset consists of %d faces" % n_samples)

```

Actividades a realizar para los experimentos de esta sección:

1. Obtenga una nueva representación del conjunto de datos utilizando PCA para reducir la dimensionalidad de los datos a 30. Seguidamente, utiliza el algoritmo t-SNE para convertir la representación de 30 dimensiones en un mapa bidimensional. Haga un gráfico con la varianza contenida en las 30 primeras componentes principales obtenidas.

2. Muestre la transformación obtenida en un gráfico de dispersión y asegúrese de mostrar la información de cada clase usando un color y/o símbolo a fin de poder evaluar visualmente si el gráfico de dispersión muestra alguna relación o similitudes dentro de cada clase. En este caso, para una configuración del parámetro de `perplexity = 40`, inspeccione un subgrupo de observaciones en el gráfico de dispersión y compare las imágenes originales ¿Encuentra alguna relación entre la similitud entre la transformación obtenida con t-SNE?
3. Haga uso del *widget* interactivo de IPython el cual reciba el parámetro de `perplexity` y permita visualizar la transformación obtenida con t-SNE usando distintos valores de este parámetro.

```
1 from ipywidgets import interactive
2 from sklearn.manifold import TSNE
3
4 def tsne_pipeline(param):
5     ...
6     # define your pipeline
7     tsne = TSNE(n_components=2, verbose=1, perplexity= 30, random_state=0)
8     z = tsne.fit_transform(x)
9
10 p_min = -10 #define your range
11 p_max = 10 #define your range
12 interactive(tsne_pipeline, param=(p_min, p_max))
```

- Discuta sobre la selección de este parámetro y su importancia en la transformación obtenida.
 - Describa que sucede al aumentar o disminuir el parámetro `perplexity`. Comente: ¿Qué valor de `perplexity` elegiría en este caso?
4. Compare visualmente la representación obtenida con t-SNE (a partir de representación de PCA de 30 componentes) con la representación obtenida usando PCA (con $n = 2$) para ver cuál proporciona mejor separación entre clústeres.

Referencias

- [1] Laurens Van der Maaten y Geoffrey Hinton. “Visualizing data using t-SNE.” En: *Journal of machine learning research* 9.11 (2008).
- [2] Zhihua Zhang. “The singular value decomposition, applications and beyond”. En: *arXiv preprint arXiv:1510.08532* (2015).